Which Bits Can We Drop?

Component-Wise Quantization, Mixed-Precision Selection and QAT for a Learned Image Codec

Harshith (and team) | CS6886 Final Project | November 2026

Code: `https://github.com/harshith0518/CS6886-PULSE-Quantization`

Abstract. One weight file serves eight quality settings. The usual method uses one precision for all parts of the model. That method ignores an imbalance. The encoder holds 85.6% of the parameters. The entropy model takes most of the decode time. The decoder is small, but it contains a fragile part. We measure how much each of 19 parts tolerates seven number formats. The measurements cover eight quality settings and two datasets: 69,233 results. A mixed-integer linear program (ILP) then chooses one precision for each part. The result is **21.677 MB**, against 54.688 MB in FP32: a **60.4%** size reduction at a **0.94%** worst-case quality loss. The Bøntegaard delta-rate (BD-rate) is $+0.48$ to $+0.55\%$ on three datasets. Compare this with the simple baseline: uniform INT8 gives a smaller file of 13.904 MB, but its loss is 26.18%. At the same size, the mixed assignment is **39**$\times$ better in quality. Finally we test quantization-aware fine-tuning (QAT). QAT removes **61–98%** of the damage on hand-chosen assignments. It removes **0%** on the assignment that the solver optimized. Therefore QAT repairs the cost that an assignment leaves. An optimal assignment leaves none.

1 Introduction

A learned image codec stores its weights in one file. In PULSE [1], that file serves eight quality settings and holds 14.18M parameters, or 54.688 MB in FP32. A team that wants a smaller file usually applies post-training quantization. The usual method uses one precision for the complete model, because a study of each part takes time. That method is a loss for this model. The parts are not equal. The encoder holds 85.6% of the parameters. The entropy model is only 9.4%, but it takes 73% of the decode time. The decoder is 5.0%, and one of its blocks has 4.7k weights that lose 8% at 8 bits. A larger encoder block with 2.89M weights loses only 0.5% at the same precision. Sensitivity does not follow size.

We therefore ask three questions. **(Q1)** Which parts tolerate which precision, and at which quality setting? **(Q2)** Which assignment is the smallest that stays below a quality limit? **(Q3)** Can fine-tuning recover the remaining loss, and where does it stop? A fourth question was not in our plan. It became the most useful one: why does quantization not make this codec faster?

2 Prior work

Integer quantization with one scale per output channel is standard practice [2]. A straight-through estimator supplies the gradient through the quantizer [3]. PULSE provides the codec that we study [1]. We state the allocation problem as an ILP and solve it with HiGHS [6]. For the rate-distortion comparison we use the BD-rate [4]. We make one change to that method. The original method fits a curve through four points. We have eight points, so we use monotone piecewise-cubic interpolation [5].

3 Idea and implementation

3.1 Measurement of sensitivity

We quantize one part at a time. All other parts and all activations stay in FP32. We then run the complete codec on each image at each quality setting. The plan covers seven number formats, 19 parts, two datasets and eight quality settings. The datasets are Kodak [7] for screening and CLIC Professional [8] for confirmation. We collect 69,233 rows. No run failed, and a hash check confirms each weight restoration. An FP32 gate compares the bitstream and the reconstruction with the reference. The gate must pass at all eight settings before we accept a row. Thus each difference comes from the quantized part, and from no other cause.

We use a rate-equivalent penalty to combine a quality change and a bitrate change in one number:

$$\text{penalty} = 100\left[\left(1+\tfrac{\Delta\text{bpp}}{100}\right)2^{-\Delta\text{PSNR}/\sigma}-1\right], \qquad (1)$$

where σ is the local slope of the reference curve. A value of zero means the model is indistinguishable from FP32. The ILP limits the worst setting. The worst setting is the maximum over the eight settings of the mean over the images.

The ILP adds the costs of single parts. That step is correct only if the costs are additive. We tested this condition. Divide the measured penalty of a uniform INT8 model by the sum of the 19 single-part penalties. The ratio is 0.96 to 1.05 on Kodak and 1.02 to 1.11 on CLIC. The costs are additive. One warning: do not add the worst setting of each part. That sum gives 65.3%, but the true value at one setting is 26.7%. Add the costs for one setting, then take the maximum.

3.2 The allocation problem

One weight file serves all eight settings. A different assignment for each setting is therefore not deployable. The model has one binary variable for each pair of a part and a format. Each variable is shared by all settings. The model has one quality constraint for each setting:

$$\begin{aligned} \min\ & \textstyle\sum_{c,f} \text{size}(c,f)\, x_{c,f} \\ \text{s.t.}\ & \textstyle\sum_{c,f} \text{pen}(c,f,q)\, x_{c,f} \le \epsilon \quad \forall q \in \{0,\dots,7\} \\ & \textstyle\sum_{f} x_{c,f} = 1 \quad \forall c. \end{aligned}$$

The objective is the number of bytes. It is not latency. We have no measured latency table for each format on this GPU. An objective that used such a table would optimize data that we invented. We also solve a greedy baseline for comparison.

3.3 Quantization-aware fine-tuning

QAT applies quantized convolutions with a straight-through gradient. It then minimizes distortion and holds the bitrate at its pre-quantization value. A fixed rate penalty failed in both directions. A weak penalty lets the model move down the rate-distortion curve. A strong penalty destroys quality to obey the rate target. We use an augmented Lagrangian instead. The multiplier μ comes from the measured rate error on a fixed validation set:

$$\mathcal{L} = \frac{D}{D_{\text{ref}}} + \mu\left(\frac{R}{R_{\text{ref}}}-1\right), \qquad \mu \leftarrow \text{clip}(\mu + \alpha\, e_{\text{rate}}).$$

We freeze the entropy parts. The bundle contains integer cumulative distribution function (CDF) tables. If the entropy weights

Table 1: Measured operating points. Values are worst-setting penalty (%), and a lower value is better. FP32 is 54.688 MB. The BD-rate of the champion is $+0.48/+0.55/+0.49\%$.

Point	MB	Save	Kodak	CLIC
1% champion	**21.677**	-60.4%	**0.67**	**0.85**
1.8% variant	18.664	-65.9%	1.27	1.60
INT8 tier + QAT	**14.401**	-73.7%	1.85	2.87
uniform INT8	13.904	-74.6%	26.18	56.70

Table 2: QAT at 12,000 steps. Values are worst-setting penalty (%) before and after. The last row used 8,000 steps.

Tier	MB	Before	After	Removed
INT8, 16 comps	14.401	8.61	**3.36**	61.0%
INT4, 2 blocks	13.572	119.18	**20.53**	82.8%
INT4, 4 blocks	11.369	1771.71	**41.84**	97.6%
ILP 2% assignment	18.389	1.936	1.936	**0.0%**

move, those tables no longer describe the weights, and the encoder and the decoder disagree. The saved checkpoint must be better than its own start. If it is not, we keep the start weights. A bad run therefore cannot ship a worse model. Training uses 100 DIV2K images [9]. These images are disjoint from all test sets. We run 12,000 steps on 512-pixel crops, which takes about 100 minutes for one tier on one 4 GB laptop GPU (PyTorch 2.6, CUDA 12.4, HiGHS via SciPy). This work uses 2.2% of the optimizer steps and 0.2% of the sample budget of the pretraining [1]. Thus it is a repair operation, not a new training.

4 Evaluation

4.1 Comparison with the baseline

Table 1 gives three operating points. We measure each point on three test sets at all eight settings, against the same FP32 reference. The solver predicted 0.9715% for the champion, and the measurement gave 0.94%. Thus the prediction is conservative on each dataset. We did not use Set14 [10] for fitting, and the result holds there. This is the best evidence that the assignment transfers.

The last row is the important comparison. Uniform INT8 gives a smaller file than our champion. Its loss is 39 times larger on Kodak and 67 times larger on CLIC. A uniform setting also varies 2.9 times across the datasets. The mixed assignments vary at most 1.4 times. Thus a limit on the total gives both quality and stability.

4.2 Effect of QAT

Table 2 answers Q3. QAT removes 61% to 98% of the damage on assignments that a person chose. It removes no damage on the assignment that the solver optimized. This result is the useful one. Fine-tuning replaces a good allocation; it does not extend it. The assignment decides the size of the repair. An ILP that already minimized the total damage leaves almost no damage to repair.

4.3 Absence of a speedup

No configuration was faster than FP32. We used paired interleaved benchmarking with 50 pairs for each cell. The mixed model is 4.79% slower (95% confidence interval $+1.27$ to $+9.80$). Its energy change is -0.39%, and the interval includes zero. The cause is arithmetic. A decode takes 88 ms. Entropy coding takes 80 ms of that time, and the convolutional network takes 8 ms. Quantization makes convolutions cheaper. Convolutions are 9% of a decode and 27% of a round trip. We make no speed claim for FP8, INT4 or FP4. These formats run as simulated quantization here. Only FP16 and BF16 use real kernels on this GPU, and they save energy, not time.

5 Conclusion

Granularity is the lever, not precision. At the same size, mixed precision is 39 times better than uniform precision. QAT replaces a good allocation and does not extend it. QAT removed 61% to 98% of the damage on hand-chosen assignments, and 0% on an optimal one. Both results are necessary. The second one is visible only because we measured the control case.

Limitations

(a) Our checkpoint selector gives a low estimate of the QAT gain. A probe with 6 crops rated the best INT8 model as equal to its start (-0.08%). The full evaluation shows 8.61% to 3.36%. The saved checkpoints are thus probably not the best available. (b) We quantize weights only. Activations stay in FP32, so we do not test W8A8. (c) The 4-bit formats are simulated, so we do not measure their speed. (d) The entropy coder uses one lane. This is the largest unused latency lever. (e) The QAT results use one seed and 100 images. The qualitative result holds for three tiers and two step counts. The exact percentages are single-run values.

References

[1] Jia et al. *PULSE: a learned image codec*. arXiv:2609.18602, 2026.

[2] Jacob et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR*, 2018. arXiv:1712.05877.

[3] Bengio, Léonard, Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv:1308.3432, 2013.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD-curves. *ITU-T VCEG-M33*, 2001.

[5] Fritsch, Butland. A method for constructing local monotone piecewise cubic interpolants. *SIAM J. Sci. Comput.* 5(2):300–304, 1984. DOI 10.1137/0905021.

[6] Huangfu, Hall. Parallelizing the dual revised simplex method. *Math. Prog. Comp.* 10(1):119–142, 2018. DOI 10.1007/s12532-017-0130-5.

[7] Eastman Kodak Company. Kodak PhotoCD image dataset, 1991.

[8] Toderici et al. Workshop and challenge on learned image compression. *CVPR Workshops*, 2020.

[9] Agustsson, Timofte. NTIRE 2017 challenge on single image super-resolution. *CVPR Workshops*, 2017.

[10] Zeyde, Elad, Protter. On single image scale-up using sparse-representations. *Curves and Surfaces*, 2010.

[11] Our code and results: `https://github.com/harshith0518/CS6886-PULSE-Quantization`.